

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: Framework JS: React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados

Modalidad: Presencial / Laboratorio

Duración: 90 minutos

APELLIDOS Y NOMBRES: Cruz Cruz Alexander Jhon

Paso 1: Ciclo de Render

Actualizar el estado provoca un nuevo render del componente; en desarrollo `React.StrictMode` puede provocar renderizados adicionales para detección de efectos secundarios.

- Ejemplo práctico en

`src/components/CounterCard.jsx`

```
import { useState } from 'react';

export function CounterCard() {
  const [counter, setCounter] = useState(0);

  return (
    <section className="card compact">
      <h2>Tazas vendidas</h2>
      <p>
        Controla de forma rápida el número de tazas servidas en tu tienda. Ideal para
        simular el ritmo de ventas del día.
      </p>
      <div className="counter-actions">
        <button onClick={() => setCounter((value) => value - 1)} type="button">-</button>
        <strong>{counter}</strong>
        <button onClick={() => setCounter((value) => value + 1)}
          type="button">+</button>
      </div>
    </section>
  );
}
```

y en el uso de estado en `src/App.jsx`.

```
import { CounterCard } from './components/CounterCard.jsx';
import { Header } from './components/Header.jsx';
import SearchPanel from './components/SearchPanel.jsx';
import { useTheme } from './context/ThemeContext.jsx';
```

```

import { useFetch } from './hooks/useFetch.js';
import { products } from './data/products.js';
import { useState } from 'react';

const USERS_API = 'https://jsonplaceholder.typicode.com/users';

function App() {
  const { theme } = useTheme();
  const { data: users, loading, error } = useFetch(USERS_API);
  const [cart, setCart] = useState([]);
  const [showCart, setShowCart] = useState(false);

  ))}
</ul>
<div style={{ marginTop: 12 }}>
  <strong>Total: ${cart.reduce((s, p) => s + p.price,
0).toFixed(2)}</strong>
</div>
</aside>
  )}
</div>
</main>
);
}

export default App;

```

Paso 2: Fetch de API

Encapsular la lógica de petición (loading, data, error) en un hook reusable y manejar cancelación con `AbortController`.

Implementado en `src/hooks/useFetch.js` y consumido desde `src/App.jsx` (variable `USERS\_API`).

```

import { useEffect, useReducer } from 'react';

const initialState = {
  data: [],
  loading: true,
  error: null,
};

function fetchReducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':
      return { ...state, loading: true, error: null };
    case 'FETCH_SUCCESS':

```

```

        return { data: action.payload, loading: false, error: null };
      case 'FETCH_ERROR':
        return { data: [], loading: false, error: action.payload };
      default:
        return state;
    }
  }
}

export function useFetch(url) {
  const [state, dispatch] = useReducer(fetchReducer, initialState);

  useEffect(() => {
    const controller = new AbortController();

    async function loadData() {
      dispatch({ type: 'FETCH_START' });

      try {
        const response = await fetch(url, { signal: controller.signal });
        if (!response.ok) {
          throw new Error('No se pudo obtener la información de la API');
        }
        const json = await response.json();
        dispatch({ type: 'FETCH_SUCCESS', payload: json });
      } catch (error) {
        if (error.name !== 'AbortError') {
          dispatch({ type: 'FETCH_ERROR', payload: error.message });
        }
      }
    }

    loadData();

    return () => {
      controller.abort();
    };
  }, [url]);

  return state;
}

```

Paso 3: useContext para tema global / estado de sesión

Compartir estado global (tema o estado de sesión) sin pasar props manualmente por árbol de componentes.

src/context/ThemeContext.jsx` exporta `ThemeProvider` y `useTheme()`; consumido en src/components/Header.jsx y en src/App.jsx.

```
import { createContext, useContext, useMemo, useState } from 'react';

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme((currentTheme) => (currentTheme === 'light' ? 'dark' : 'light'));
  };

  const value = useMemo(() => ({ theme, toggleTheme }), [theme]);

  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
}

export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme debe usarse dentro de ThemeProvider');
  }
  return context;
}
```

Paso 4: Optimizar render:

4.1. useMemo

Envolver cálculos o filtros costosos en `useMemo` para evitar recalculer en cada render cuando sus dependencias no cambiaron.

Filtro de catálogo en `src/components/SearchPanel.jsx` filtra por texto, categoría y precio; `useMemo` evita recalculer cuando `products`, `values.search` y `minimumPrice` no cambian.

```
import { memo, useCallback, useMemo, useRef, useState } from 'react';
import { Search } from 'lucide-react';
import { useForm } from '../hooks/useForm.js';

function SearchPanel({ products, onAddToCart }) {
  const inputRef = useRef(null);
  const [minimumPrice, setMinimumPrice] = useState(0);
  const [category, setCategory] = useState('Todos');
  const { values, handleChange, resetForm } = useForm({ search: '' });

  const categories = useMemo(() => [
    'Todos',
    ...Array.from(new Set(products.map((product) => product.category))),
  ], [products]);
```

```
const filteredProducts = useMemo(() => {
  const text = values.search.toLowerCase().trim();
  return products
    .filter((p) => p.price >= minimumPrice)
    .filter((p) => category === 'Todos' || p.category === category)
    .filter((p) => p.name.toLowerCase().includes(text) ||
p.description.toLowerCase().includes(text) ||
p.category.toLowerCase().includes(text));
}, [products, values.search, minimumPrice, category]);
```

4.2. Estabilizar callbacks: `useCallback`

Memorizar funciones que se pasan como props para evitar re-renderes innecesarios de componentes hijos o para mantener referencias estables en dependencias.

`focusInput` y `clearFilters` en `src/components/SearchPanel.jsx` usan `useCallback`.

```
const focusInput = useCallback(() => {
  inputRef.current?.focus();
}, []);

const clearFilters = useCallback(() => {
  resetForm();
  setMinimumPrice(0);
  focusInput();
}, [resetForm, focusInput]);
```

4.3. useRef para DOM

`useRef` mantiene una referencia mutable a nodos DOM sin provocar renders; ideal para `focus()` en inputs.

el botón hero en `src/App.jsx` invoca el foco sobre el input.

```
<section className="hero card">
  <div>
    <span className="badge">Lo más pedido</span>
    <h2>Tu tienda de café favorita, con pedidos rápidos y sabores
premium.</h2>
    <p>Explora nuestro catálogo de espresso, leches especiales y bebidas de
temporada con entrega express.</p>
    <div className="hero-actions">
      <button className="theme-button" onClick={() => {
document.querySelector('input[name="search"]')?.focus(); }}>Ver el menú</button>
      <button className="secondary">Promociones</button>
    </div>
  </div>
  <div className="hero-illustration">
    <div className="hero-card">30% OFF</div>
```

```

        <div className="hero-card hero-card--soft">Leche de almendra
disponible</div>
    </div>
</section>

```

Paso 5: Lógica repetitiva en Hook personalizado

Extraer lógica repetitiva (peticiones, manejo de formularios) a hooks personalizados aumenta la reutilización y testabilidad.

src/hooks/useFetch.js (peticiones con `loading`/`error`) y `src/hooks/useForm.js` (manejador de inputs, `handleChange`, `resetForm`).

```

import { useEffect, useReducer } from 'react';

const initialState = {
  data: [],
  loading: true,
  error: null,
};

function fetchReducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':
      return { ...state, loading: true, error: null };
    case 'FETCH_SUCCESS':
      return { data: action.payload, loading: false, error: null };
    case 'FETCH_ERROR':
      return { data: [], loading: false, error: action.payload };
    default:
      return state;
  }
}

export function useFetch(url) {
  const [state, dispatch] = useReducer(fetchReducer, initialState);

  useEffect(() => {
    const controller = new AbortController();

    async function loadData() {
      dispatch({ type: 'FETCH_START' });

      try {
        const response = await fetch(url, { signal: controller.signal });
        if (!response.ok) {
          throw new Error('No se pudo obtener la información de la API');
        }
        const json = await response.json();

```

```

        dispatch({ type: 'FETCH_SUCCESS', payload: json });
    } catch (error) {
        if (error.name !== 'AbortError') {
            dispatch({ type: 'FETCH_ERROR', payload: error.message });
        }
    }
}

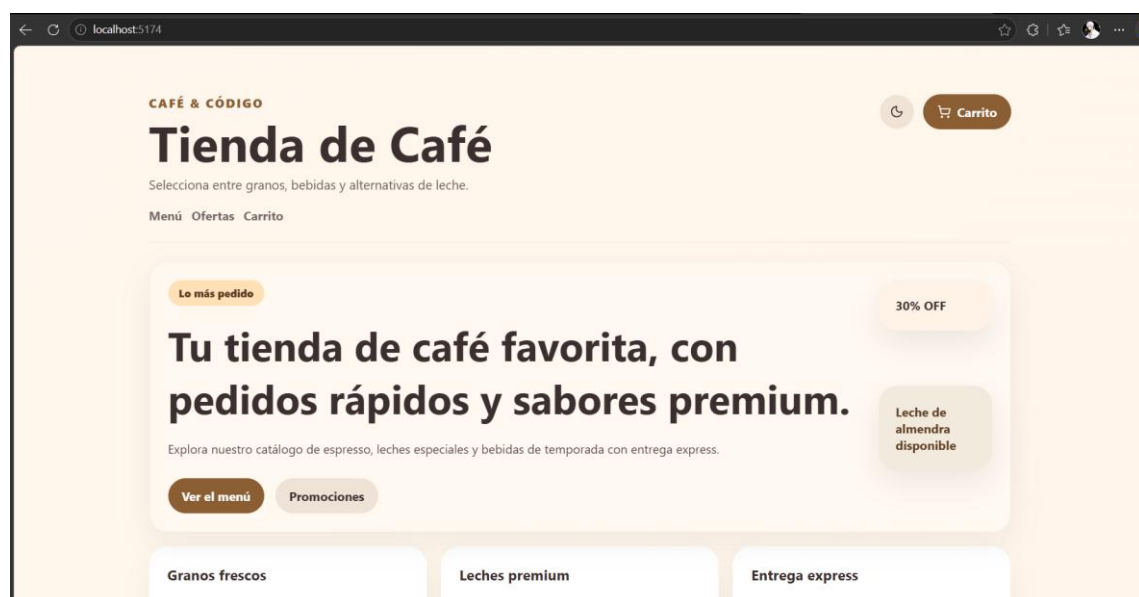
loadData();

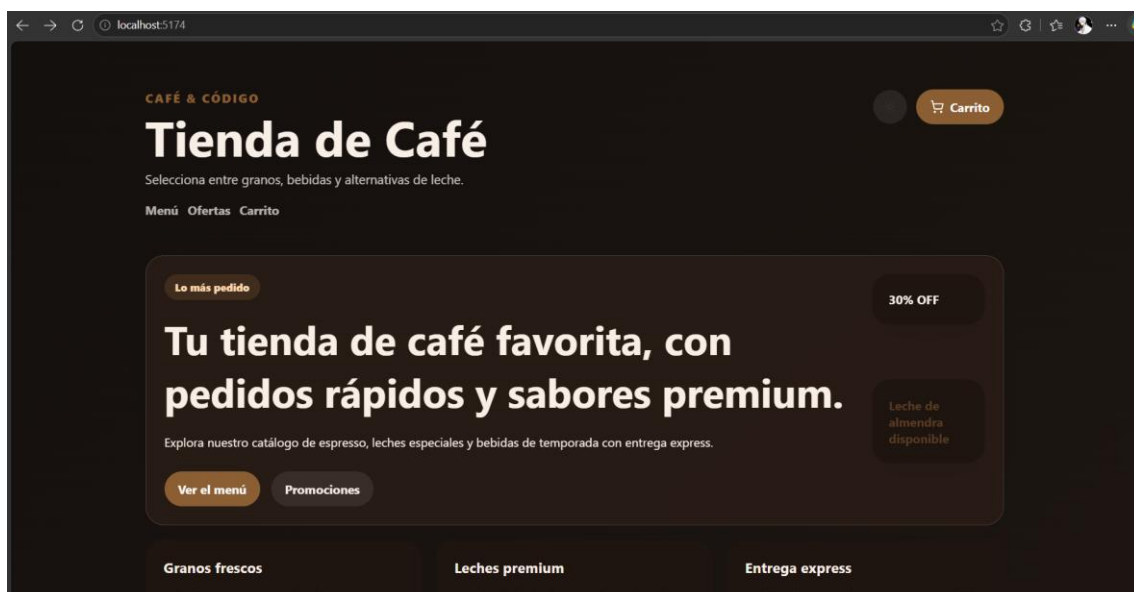
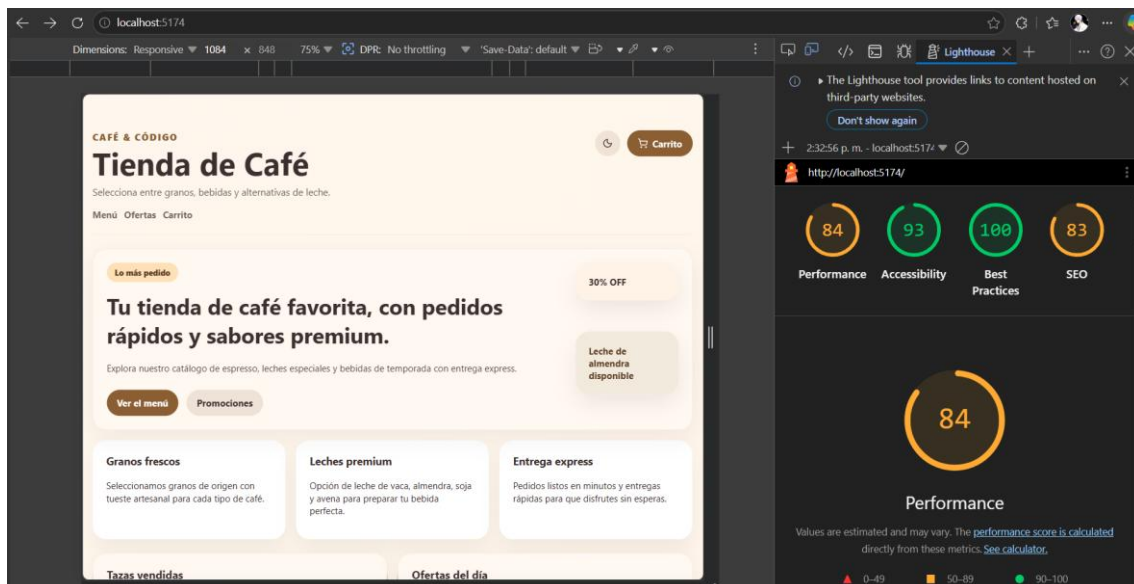
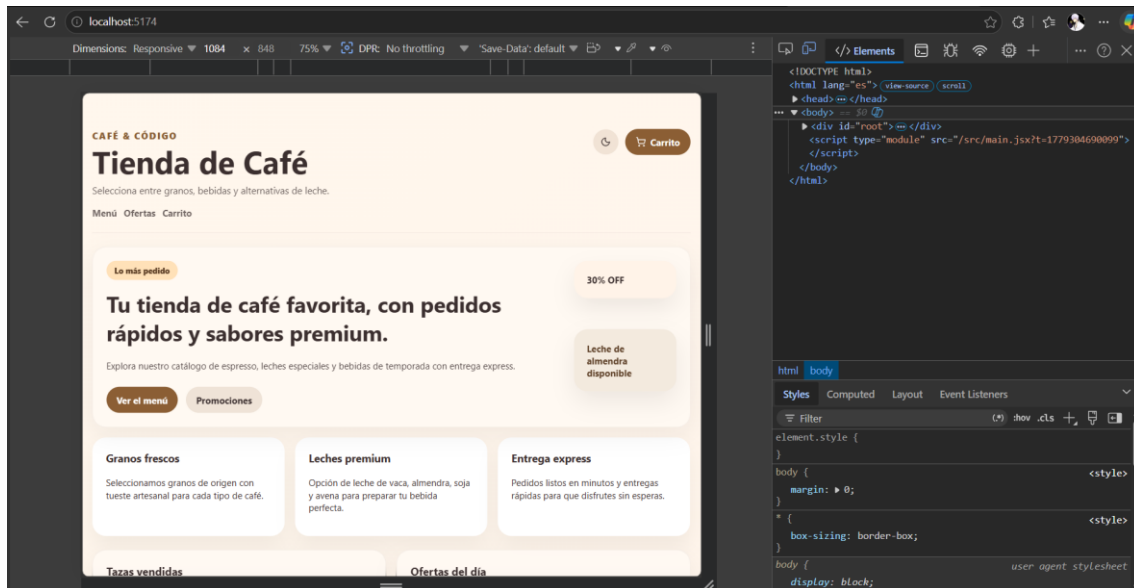
return () => {
    controller.abort();
};
}, [url]);

return state;
}

```

Resultado de la página:





Link de github: [Ichimaru465/S7-practica-Cruz](https://github.com/Ichimaru465/S7-practica-Cruz)